

Essay Perbaikan Tugas Proyek 1
IF4070 Representasi Pengetahuan dan Penalaran
Ontologi dan Sistem Berbasis Pengetahuan



Disusun Oleh:

Juan Alfred Widjaya

13522073

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

1. Penjelasan Perbaikan

Pada Tugas Proyek 1 IF 4070 saya berkontribusi dalam pembuatan code dan dalam penyusunan laporan. Masing masing kami di setiap subbab memiliki perannya masing masing untuk saling melengkapi kekurangan satu sama lain.

Saya Juan Alfred Widjaya mengakui dalam bab IV.2 yang berjudul **Keterlibatan logika deskripsi** dan bab IV.3 yang berjudul **Keterbatasan ontologi dan sistem berbasis pengetahuan** menggunakan AI generatif untuk mengubah draft yang berisi penjelasan singkat di masing masing subbab menjadi paragraf paragraf yang rapi supaya antar kalimat saling terhubung dan terstruktur. Namun hal ini membuat hasil laporan saya menjadi seperti AI full generated sehingga terdapat banyak seperti kata kata yang terlalu formal dan juga pola yang sangat “AI generatif”.

Perbaikan ini akan difokuskan dengan memperbaiki penulisan di bab IV.2 dan di bab IV.3

2. Isi Perbaikan

Berikut isi perbaikan dari saya. (berdasarkan laporan utama awal LaporanRPP_BMW.pdf)

IV.2 Keterlibatan logika deskripsi

Sistem ini dirancang mengikuti prinsip Logika Deskripsi atau DL meskipun secara teknis dibuat dengan Prolog yang berbasis First-Order Logic atau FOL. Melalui paradigma rule-based reasoning, sistem ini mampu meniru perilaku DL dengan cukup baik. Keterlibatan logika deskripsi ini terlihat jelas dalam struktur sistem.

Konsep TBox dan ABox dipisahkan secara jelas dalam implementasi ini. TBox diwujudkan lewat aturan Prolog contohnya `electric_car(M) :- has_powertrain(M, electric)` yang mengatur definisi kelas. ABox diwujudkan lewat fakta individu contohnya `has_powertrain(bmw_i7_xdrive60, electric)` yang menyimpan data instans.

Pemisahan ini penting agar satu set aturan TBox bisa dipakai untuk banyak data ABox sehingga sistem lebih mudah dikelola.

Berbagai konstruktor Logika Deskripsi juga diterjemahkan ke dalam sintaks Prolog. Dalam penulisan aturan, konjungsi ditulis dengan tanda koma. Contohnya `offroad_capable_suv(M) :- has_body_type(M, suv), has_feature(M, xdrive)` yang artinya offroad capable suv adalah mobil yang berjenis SUV dan memiliki fitur xDrive sekaligus. Disjungsi ditulis menggunakan semicolon

Sistem ini mendukung klasifikasi implisit. Jika sebuah model punya ciri yang sesuai dengan definisi kelas maka ia otomatis masuk ke kelas itu. Misal karena `bmw_m3g80` punya spek `m_series` sistem langsung paham bahwa itu adalah mobil berperforma tinggi. Proses inferensi ini berjalan otomatis tanpa perlu deklarasi manual yang berlebihan.

Walaupun begitu Prolog tetap bukan bahasa Logika Deskripsi yang sempurna. Ada keterbatasan seperti tidak adanya hierarki kelas bawaan serta sulitnya menerapkan negasi kuat atau batasan batasan tertentu. Namun tantangan ini bisa diatasi dengan perancangan aturan yang matang dan pembatasan domain yang sesuai. Jadi Prolog tetap menjadi alat yang sangat berguna untuk tujuan ini.

IV.3 Keterbatasan ontologi dan sistem berbasis pengetahuan

Sistem ini memiliki keterbatasan karena tidak adanya pembedaan formal antara subkelas dan kelas independen. Dalam implementasi Prolog saat ini atribut seperti `m_performance` dan `m_series` hanya dianggap sebagai nilai biasa dan bukan kelas dengan hubungan pewarisan. Sistem tidak bisa langsung tahu hubungan antar kelas tanpa aturan yang ditulis berulang kali. Hal ini tentu menambah kerumitan dan mengurangi efisiensi kerja sistem yang seharusnya bisa lebih canggih dalam menangani hierarki.

Sistem ini beroperasi dengan prinsip Closed-World Assumption yang menganggap semua nilai atribut seperti `body_type` atau `powertrain` sudah diketahui dan terbatas jumlahnya. Cara ini memang aman untuk data yang stabil tetapi akan jadi masalah jika muncul jenis baru karena sistem tidak siap menerimanya secara otomatis. Setiap ada informasi baru daftar enum harus diperbarui secara manual agar data tersebut bisa dikenali.

Ada juga masalah pada representasi properti fungsional di mana satu individu seharusnya cuma punya satu nilai spesifik misalnya satu mobil hanya punya satu jenis `powertrain`. Sayangnya Prolog tidak otomatis memaksakan aturan ini sehingga data ganda bisa saja masuk. Inkonsistensi ini hanya bisa dicegah jika dibuat aturan penangkal konflik secara eksplisit seperti kode `:- has_powertrain(M, P1), has_powertrain(M, P2), P1 \= P2` untuk mendeteksi pelanggaran tersebut.

Untuk urusan klasifikasi yang rumit atau kombinasi fitur yang kompleks sistem ini mengharuskan penulisan aturan manual satu per satu. Sistem tidak mendukung query komposisional yang dinamis sehingga klasifikasi canggih tidak bisa dibuat secara dadakan seperti dalam lingkungan DL formal. Keterbatasan ini membuat fleksibilitas sistem jadi berkurang terutama saat butuh pencarian data secara ad-hoc.

Ontologi yang ada sekarang bersifat statis dan tidak memperhitungkan dimensi waktu atau evolusi model. Tidak ada pembedaan tahun produksi atau generasi semisal antara BMW 3 Series F30 dan G20 padahal spesifikasi teknis sering berubah seiring waktu. Tanpa adanya dimensi temporal ini risiko kesalahan klasifikasi untuk model lama versus model baru menjadi sangat besar.

Sistem berbasis Prolog ini sangat bergantung pada pengetahuan yang dimasukkan secara eksplisit ke dalamnya tanpa kemampuan belajar pola sendiri. Sebagai contoh walaupun semua data SUV BMW yang ada memiliki fitur `xdrive` sistem tidak akan berani menyimpulkan bahwa SUV baru pasti AWD kecuali fakta

has_drivetrain(ModelBaru, awd) dimasukkan secara manual. Hal ini membuat campur tangan manusia mutlak diperlukan terus-menerus agar pengetahuan sistem tetap relevan.